
A Comparative Evaluation of Multi-Armed Bandit Architectures for Personalised News Recommendations

Muhammad Fariz Bin Mohammad Yusri Ng Jun Wei Rasyiqah binte Shaik Mohamed Salim Tan Hua Swen
Teo Boon Khai, Marcus

Abstract

Online news platforms struggle with content discovery due to the rapid decay of article relevance and the user cold-start problem. This paper evaluates the efficacy of contextual multi-armed bandits in optimising news recommendations by leveraging user and article features. We compare the performance of fixed ϵ -greedy, decaying ϵ -greedy, LinUCB, and Linear Thompson Sampling strategies using a synthetic reward model to overcome the data efficiency limitations of standard replay evaluation. The algorithms are benchmarked on Total/Average Reward and Total/Average Regret to quantify the trade-off between exploration and exploitation. Our results demonstrate that LinUCB's uncertainty-based exploration achieves superior convergence and lower cumulative regret compared to probability-based baselines.

1. Introduction

1.1. Problem Statement

The exponential growth of digital content presents a significant information retrieval challenge for online news platforms, where article relevance decays rapidly. Traditional recommendation systems, such as Collaborative Filtering (CF) and Matrix Factorisation (MF), are fundamentally limited by their static training cycles and inability to adapt to shifting user preferences in real-time. While Multi-Armed Bandits (MABs) offer a dynamic alternative, the high cardinality of news catalogues, often exceeding tens of thousands of items, poses a significant challenge for efficient exploration. Consequently, there is a critical need to evaluate how contextual bandit algorithms perform when the action space is abstracted into thematic categories, balancing the granularity of recommendations with the necessity for rapid convergence in a high-volume data environment.

1.2. Background

Multi-Armed Bandits (MABs) address these limitations by framing recommendations as an iterative decision-making

process under uncertainty. Unlike static models, MABs employ an exploration-exploitation trade-off to maximise long-term rewards. In the context of news, Contextual Bandits extend this framework by leveraging side information such as user demographics and article categories. This study focuses on four contextual strategies: ϵ -greedy, decaying ϵ -greedy, LinUCB, and Linear Thompson Sampling. LinUCB uses ridge regression with uncertainty bounds, while Linear Thompson Sampling samples from a posterior over arm-specific linear reward parameters to select articles that maximise the expected Click-Through Rate (CTR). Standard offline testing usually relies on Replay Evaluation, where we only look at rounds where our model matches the original historical record. However, if our model suggests a category that was not in the original logs, we have no way of knowing if the user would have clicked it, and that data is wasted. To solve this, we use a Reward Model to act as a synthetic environment. This simulator provides feedback for any choice the model makes, allowing us to accurately measure how quickly each algorithm learns and how much 'Regret' it accumulates over time.

1.3. Objective

The primary objective of this research is to perform a comparative study of contextual bandit exploration strategies in a simulated offline environment. By analysing fixed and decaying ϵ -greedy, LinUCB, and Linear Thompson Sampling, we aim to evaluate the fundamental trade-offs between probability-based, uncertainty-based, and posterior-sampling exploration policies. While CTR measures final performance, we utilise Cumulative and Average Regret as diagnostic metrics to quantify the 'cost of learning'. This approach allows us to investigate how different mathematical frameworks, ranging from simple stochastic selection to rigorous confidence bounds, can impact the speed of convergence and the efficiency of the exploration-exploitation trade-off in dynamic news recommendation.

2. Dataset

2.1. Overview

To evaluate the proposed contextual bandit algorithms, we utilise the Microsoft News Dataset (MIND) (Wu et al., 2020), a large-scale benchmark derived from anonymised user logs of the Microsoft News portal. We specifically employ the MIND-small subset, containing approximately 50,000 users and their interactions across various news categories. Each impression log serves as a decision round where the algorithm recommends an article (arm) from a candidate set based on user history and article metadata. This dataset is particularly suitable for our study as it provides the rich side information required to construct high-dimensional context vectors for LinUCB, Linear Thompson Sampling, and contextual ϵ -greedy variants.

2.2. Pre-processing

The raw MIND dataset underwent several stages of transformation to ensure computational feasibility and model stability. We first established a master dataset, `events_df`, from which we derived `featured_df` (containing engineered user and article metadata) and `train_df` (the feature set used to train the reward model).

To ensure a stable foundation for the Multi-Armed Bandit (MAB) to learn, we performed targeted row reduction. Rather than random sampling, we selected users with a minimum of 150 candidate impressions. This filtering reduced the dataset from 6 million to approximately 500,000 rows, and the number of arms from 17 to 16 (“middleeast” category only existed in users with less than 150 candidate impressions). This choice was made to provide a sufficient “horizon” for the MAB to initialise across all available arms without the noise typical of short-lived user sessions.

Furthermore, we addressed the high cardinality of the MIND dataset, which contains 51,282 unique articles, by abstracting the action space. As a practical limitation, treating every article as an individual arm is computationally prohibitive and yields very sparse data. We instead utilised the 16 news categories as our arms. Both the reward model and algorithm recommendations operate at the category level.

2.3. Feature Engineering

To provide the contextual information necessary for both the reward model and the bandit policies, we transformed raw interaction logs into a high-dimensional feature set, `featured_df`. We focused on four primary predictors designed to capture the alignment between user history and the proposed action:

- **Semantic Category Similarity:** This feature represents the conceptual distance between a user’s historical

interests and a candidate category. We constructed “category profiles” by averaging the 100-dimensional Knowledge Graph (KG) embeddings of all Wikidata QIDs associated with a category’s articles. We similarly generated “user profiles” from their reading history. By calculating the cosine similarity between these two vector spaces, we created an “arm-swappable” feature. This is a critical engineering step for offline simulation, as it allows the model to estimate rewards for counterfactual categories which were not present in the original logs.

- **Historical Engagement (History Length):** A numerical feature tracking the total volume of a user’s past interactions, serving as a surrogate for user maturity and the density of their preference profile.
- **Category Match Ratio:** A measure of a user’s broad interest, calculated as the proportion of historical clicks that fall within the same category as the current suggested arm.
- **Categorical Context:** The thematic category of the suggested article, treated as a nominal feature to capture global popularity trends across the 16 available arms.

These features collectively allow our models to map the state-action pair (s, a) into a feature space where the LinUCB and Linear Thompson Sampling algorithms can perform linear regression to estimate expected rewards.

2.4. State Representation and Feature Mapping

To enable the contextual bandit algorithms to generalise across users and news categories, we map each decision instance into a d -dimensional feature space ($d = 18$). This vector, $x_{a,t}$, serves as the comprehensive numerical representation of the state-action pair at each time step. We distinguish between two types of contextual information within our vector construction:

- **Stationary User Context:** These features remain constant across all candidate arms for a specific impression. They include the user’s total `history_length` and a 16-dimensional profile of their historical click distribution (`hist_pct`). This provides the algorithm with a “static” snapshot of the user’s past engagement levels and topical interests.
- **Dynamic Arm-Context Interaction:** These features are unique to the relationship between the user and a specific candidate arm (category). Specifically, the `semantic_similarity_category` is dynamically recalculated for every potential arm pull. By performing a lookup against precomputed Knowledge Graph embeddings for the candidate category

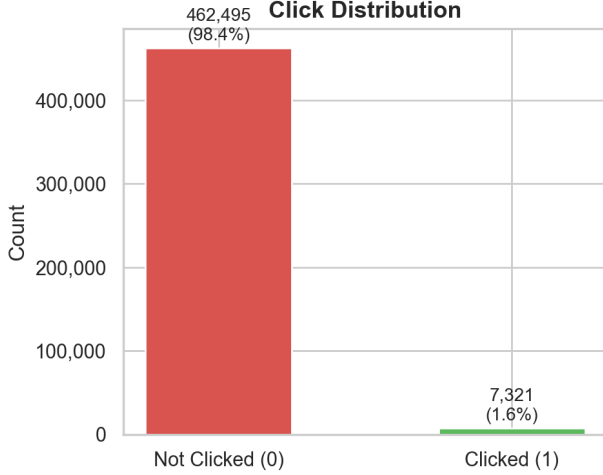


Figure 1. Overall click distribution

and comparing it to the user’s history, the simulation generates a specific distance metric for each available choice.

This dynamic vectorisation is critical for the functioning of contextual policies. By concatenating the stationary user profile with arm-specific similarity scores, we construct a feature vector that allows the algorithms to differentiate between the potential utility of the 16 available categories based on their alignment with the user’s historical context.

3. Exploratory Data Analysis

3.1. Raw Event Stream

3.1.1. CLICK DISTRIBUTION AND OVERALL CTR

The dataset exhibits a pronounced class imbalance that is characteristic of real-world news recommendation logs. Of the 469,816 impression-level events retained after the impression-count filter, 462,495 (98.4%) are non-clicks and only 7,321 (1.6%) are clicks (Figure 2), yielding an overall CTR of approximately 1.6%. This severe imbalance has two practical implications: it motivates the use of reward signals that are relative rather than absolute (as in the MAB regret framework), and it underscores why raw accuracy would be a misleading evaluation metric.

3.1.2. CTR BY CATEGORY

Figure 2 plots the per-category CTR against the overall baseline of 1.6% (dashed line). Several categories stand out. *TV* and *Health* both exceed 2.0%, roughly double the average, suggesting that users who are shown content in these categories are meaningfully more likely to engage.

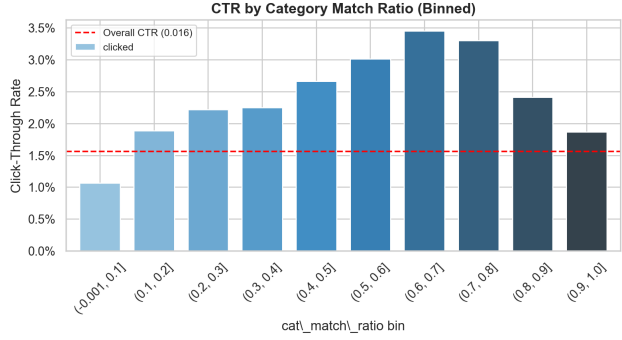


Figure 2. CTR by top-level article category

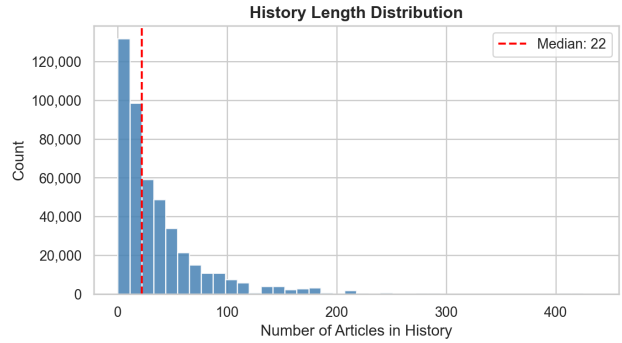


Figure 3. Distribution of user history lengths

Conversely, *Weather*, *Kids*, and *Northamerica* sit well below the baseline. These heterogeneous CTRs confirm that the category of a recommended article is a strong signal and justify treating category selection as the core bandit problem.

3.1.3. USER HISTORY LENGTH

The history length distribution (Figure 3) is heavily right-skewed: the median history length is 38 articles, but a small fraction of users have histories exceeding 300 articles. The majority of users cluster in the 0–100 range, indicating that most individuals have limited prior interaction data available at inference time. This skew is partly a consequence of our pre-processing choice to retain only users with at least 150 impression candidates, which biases the sample towards more active users.

3.2. Post Feature Engineering

3.2.1. CATEGORY MATCH RATIO AND CTR

The `cat_match_ratio` feature captures the fraction of articles in a user’s reading history whose top-level category matches the category of the currently shown article.

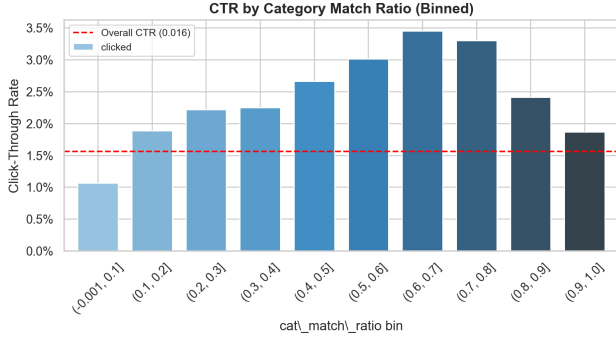


Figure 4. Binned CTR by `cat_match_ratio`

The binned CTR plot (Figure 4) reveals a clear monotonic relationship: CTR rises steadily from about 1.0% at the lowest match ratios to over 3.0% for impressions where the shown category matches more than 70% of the user’s history. This strong signal validates `cat_match_ratio` as a useful contextual feature: users are substantially more likely to click on content that aligns with their demonstrated category preferences.

3.3. Summary of EDA Findings

The EDA establishes three key takeaways that directly motivate our modelling approach:

1. **Category identity.** Per-category CTRs vary by up to a factor of two around the mean, confirming that the choice of arm (category) has a material impact on user engagement.
2. **Interest alignment is predictive.** `cat_match_ratio` shows a clear monotonic relationship with CTR, making it a strong contextual signal for LinUCB and the logistic regression oracle.
3. **Class imbalance is severe.** With a 98.4%/1.6% split, evaluation metrics must be chosen carefully. We therefore report cumulative reward, normalised regret, and ROC-AUC rather than raw accuracy.

4. Methodology

4.1. Reward Model

A logistic regression model is used as the reward model. Its role is not to choose actions directly, but to estimate the click probability of a category given a user context. This estimated probability is then used to simulate stochastic rewards via a binomial draw.

The input features for the reward model are the four primary predictors identified in the feature engineering section

previously:

- History Length
- Semantic Category Similarity
- Category match ratio
- Category context shown

A particularly important modelling decision is the use of interaction terms through second-degree polynomial feature expansion with `interaction_only=True`. This allows the model to learn patterns such as whether users with a high sports-history proportion are especially likely to click when the shown category is sports. Essentially, the model captures not only user preference and category popularity separately, but also their interaction.

The reward model was trained using an 80/20 chronological train-test split: train dataset (375,852) and test dataset (93,964). To account for the severe class imbalance, the logistic regression uses `class_weight='balanced'` which assigns a higher penalty to misclassification of the minority class (clicked = 1) and prevents the model from being biased toward predicting the majority class. We obtain a test performance of **AUC = 0.6598**. This indicates that the reward model has learned meaningful click signals and performs better than random ranking. This is a reasonable outcome given the sparsity and noisiness of click behaviour.

4.2. Algorithms

Four contextual bandit algorithms are implemented and compared. The MAB algorithms were run against the reward model only on the test dataset (93,964 rows). Although this limits the number of timesteps available for the MABs to learn and converge, it ensures a fairer offline evaluation by preventing data leakage. The reward model was trained on the train dataset, and running it again on the full dataset will cause it to be more accurate on the train dataset than the test dataset. That means the simulated environment may be artificially “clean”, hence more predictable. The bandit policies may appear to learn faster and accumulate lower regret than they would on unseen impressions.

Non-decaying ϵ -greedy

This algorithm explores with a constant probability ϵ and otherwise exploits the arm with the highest predicted reward. It uses per-arm ridge regression updates based on observed rewards. The ϵ -greedy baseline was adapted from the contextual bandit news recommendation in Li et al. (Li et al., 2010), estimating each arm’s reward as $\hat{\theta}_a^\top x_{t,a}$ (linear model) and selecting the category with the highest predicted contextual reward during exploitation.

Decaying ϵ -greedy

Algorithm 1 Oracle Reward Model

Input: reward model $f(\cdot)$, context c_t , arms $\mathcal{A} = \{1, \dots, K\}$
for each arm $a \in \mathcal{A}$ **do**
 Build $x_{t,a}$ and compute $\hat{p}_{t,a} = f(x_{t,a})$
end for
 $a_t^* = \arg \max_{a \in \mathcal{A}} \hat{p}_{t,a}$, $r_t^* = \max_{a \in \mathcal{A}} \hat{p}_{t,a}$
If arm a_t is played, observe $r_t \sim \text{Bernoulli}(\hat{p}_{t,a_t})$
 $\text{regret}_t = r_t^* - \hat{p}_{t,a_t}$

Algorithm 2 Contextual ϵ -Greedy

Input: ϵ, λ , arms $\mathcal{A} = \{1, \dots, K\}$
Initialize $A_a = \lambda I_d$, $b_a = \mathbf{0}_d$ for all $a \in \mathcal{A}$
for round $t = 1$ **to** T **do**
 for each arm $a \in \mathcal{A}$ **do**
 Build $x_{t,a}$, compute $\hat{\theta}_a = A_a^{-1} b_a$, $\hat{\mu}_{t,a} = \hat{\theta}_a^\top x_{t,a}$
 end for
if $\text{Uniform}(0, 1) < \epsilon$ **then**
 Select random arm a_t
else
 Select $a_t = \arg \max_{a \in \mathcal{A}} \hat{\mu}_{t,a}$
end if
Observe reward r_t
Update $A_{a_t} = A_{a_t} + x_{t,a_t} x_{t,a_t}^\top$, $b_{a_t} = b_{a_t} + r_t x_{t,a_t}$
end for

This variant begins with a higher exploration rate and gradually reduces exploration over time, down to a minimum threshold. It is intended to encourage more exploration early in learning and more exploitation later.

LinUCB

The LinUCB implementation follows the algorithm proposed and implemented by Li et al. (Li et al., 2010) in a similar setting. LinUCB estimates a linear reward model for each arm and augments the predicted reward with an uncertainty bonus. This confidence term encourages exploration of arms with greater uncertainty and tends to be more targeted than random exploration.

Linear Thompson Sampling

The Linear Thompson Sampling implementation follows the linear contextual bandit formulation proposed by Agrawal and Goyal (Agrawal & Goyal, 2013), and later built on by Akshayaa (Magesh, 2019), where an arm-specific posterior distribution over linear reward parameters is maintained and sampled from at each decision round. At every round, it samples a coefficient vector from each arm's posterior distribution and scores each arm using the sampled parameter vector, then selects the arm with the highest sampled reward. The posterior sampling variance scale is controlled by $v = 0.1$ in our implementation.

Algorithm 3 LinUCB

Input: α , arms $\mathcal{A} = \{1, \dots, K\}$
Initialize $A_a = I_d$, $b_a = \mathbf{0}_d$ for all $a \in \mathcal{A}$
for round $t = 1$ **to** T **do**
 for each arm $a \in \mathcal{A}$ **do**
 Build $x_{t,a}$, compute $\hat{\theta}_a = A_a^{-1} b_a$
 $p_{t,a} = \hat{\theta}_a^\top x_{t,a} + \alpha \sqrt{x_{t,a}^\top A_a^{-1} x_{t,a}}$
 end for
Select $a_t = \arg \max_{a \in \mathcal{A}} p_{t,a}$
Observe reward r_t
Update $A_{a_t} = A_{a_t} + x_{t,a_t} x_{t,a_t}^\top$, $b_{a_t} = b_{a_t} + r_t x_{t,a_t}$
end for

Algorithm 4 Linear Thompson Sampling

Input: posterior scale v , regularisation λ , arms $\mathcal{A} = \{1, \dots, K\}$
Initialize $A_a = \lambda I_d$, $b_a = \mathbf{0}_d$ for all $a \in \mathcal{A}$
for round $t = 1$ **to** T **do**
 for each arm $a \in \mathcal{A}$ **do**
 Build $x_{t,a}$, compute $\mu_a = A_a^{-1} b_a$
 Sample $\tilde{\theta}_a \sim \mathcal{N}(\mu_a, v^2 A_a^{-1})$
 Score $s_{t,a} = \tilde{\theta}_a^\top x_{t,a}$
 end for
Select $a_t = \arg \max_{a \in \mathcal{A}} s_{t,a}$
Observe reward r_t
Update $A_{a_t} = A_{a_t} + x_{t,a_t} x_{t,a_t}^\top$, $b_{a_t} = b_{a_t} + r_t x_{t,a_t}$
end for

4.3. Experimental Setup

The simulation is run over the full flattened interaction set. At each round:

1. A context vector is built for every arm.
 - (a) For arm a with context vector x , the estimated coefficient vector is $\hat{\theta}_a = A_a^{-1} b_a$, where A_a accumulates the outer products of observed context vectors and b_a accumulates the corresponding rewards.
2. The reward model estimates the expected reward of every arm for that user impression.
3. The highest of these is treated as the oracle benchmark for regret calculation.
4. Each bandit algorithm selects an arm.
5. A binary reward is sampled from the reward model using the chosen arm's predicted probability.
6. The algorithm updates A_a and b_a only for the chosen arm.

Table 1. Comparison of contextual bandit algorithms

	Non-Decaying ϵ -Greedy	Decaying ϵ -Greedy	LinUCB	Linear Thompson Sampling
Exploration strategy	Selects a random arm with fixed probability $\epsilon = 0.1$ at every round.	Starts with higher random exploration and decays ϵ over time to a minimum threshold.	Adds a confidence-bound bonus to the estimated reward, encouraging arms with higher contextual uncertainty.	Samples an arm-specific parameter vector from the posterior and selects the arm with the highest sampled reward.
Exploitation	Selects the arm with the highest estimated reward with probability $1 - \epsilon$.	Selects the arm with the highest estimated reward as exploration decays.	Selects the arm with the highest UCB score: $\hat{\theta}_a^\top x_a + \alpha \sqrt{x_a^\top A_a^{-1} x_a}$.	Selects the arm with the highest posterior-sampled score: $\hat{\theta}_a^\top x_a$.
Context-aware exploration	No; exploration is random.	No; only the exploration rate changes over time.	Yes; uncertainty is assessed separately for each arm and context.	Yes; uncertainty is represented through posterior sampling for each arm.
Exploration parameter	$\epsilon = 0.1$ fixed.	Initial $\epsilon = 0.3$, decay rate 0.99985, minimum 0.05.	$\alpha = 1.5$.	Posterior variance scale $v = 0.1$.
Role	Baseline for persistent exploration.	Tests whether tapering exploration improves long-run reward.	Tests confidence-bound, uncertainty-aware exploration.	Tests Bayesian posterior-sampling exploration under the same contextual setup.

5. Results & Discussion

5.1. Evaluation Metrics

Given the offline simulation setting of this study, we conducted a comprehensive evaluation of the four MAB recommender algorithms using four metrics which capture different dimensions of recommendation quality. These are summarised in Table 2 below.

Table 2. Evaluation metrics used to assess algorithm performance

Metric	Definition
Total Reward	Total number of simulated clicks accumulated across all rounds. Higher values indicate that the algorithm recommended more relevant categories overall.
Average Reward (CTR)	Total reward divided by the total number of rounds played, representing the algorithm’s overall click-through rate and measuring recommendation quality per interaction.
Total Regret	Cumulative difference between the oracle’s best possible expected reward and the expected reward of the chosen arm. Lower values indicate better arm selections relative to the theoretical optimum.
Average Regret	Total regret divided by the total number of rounds played, measuring the per-round opportunity cost of the algorithm’s decisions.

Regret is computed against a logistic regression oracle model, which always selects the arm with the highest predicted click probability. Given that the underlying logistic regression model only achieved an AUC of 0.6598, regret figures in this study are relative to an estimated upper bound rather than a true theoretical optimum. Furthermore, the oracle model achieves a high precision on non-clicked articles (0.99), but very low precision on clicked articles (0.02). This stark class imbalance reflects the inherent sparsity of positive interactions in the MIND dataset, limiting the reliability of the oracle model and should be kept in mind when interpreting regret values.

5.2. Model Performance

5.2.1. OVERALL RESULTS

Table 3 presents the aggregate performance of all four MAB algorithms over the full simulation of 94,942 held-out impressions.

Table 3. Overall Algorithm Performance

Algorithm	Total Reward	Avg Reward (CTR)	Total Regret	Avg Regret
ϵ -greedy (non-decaying)	58178	0.6128	5088.9	0.0536
ϵ -greedy (decaying)	58570	0.6169	4553.9	0.0480
LinUCB	59537	0.6271	3752.2	0.0395
Linear Thompson Sampling	56334	0.5934	6920.1	0.0729

LinUCB achieved the highest total reward and lowest cumulative regret across all impressions, followed by ϵ -greedy (decaying), ϵ -greedy (non-decaying), and Linear Thompson Sampling. Notably, Thompson Sampling accumulated the highest cumulative regret out of all four algorithms, underperforming even the non-decaying ϵ -greedy baseline. This ranking is consistent across both reward and regret metrics, hence suggesting that there is a clear performance hierarchy among the four MAB algorithms.

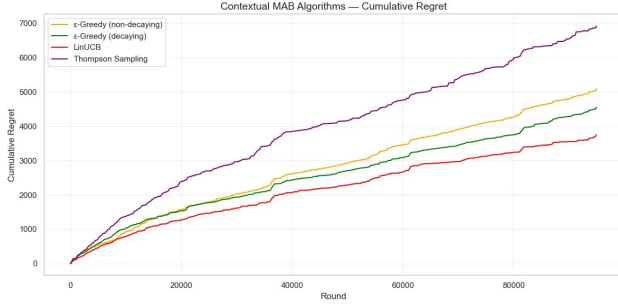


Figure 5. Cumulative regret of MAB Algorithms

5.2.2. ϵ -GREEDY: DECAYING VS NON-DECAYING

Comparing the two ϵ -greedy algorithms revealed the impact of exploration scheduling on long-run performance. The non-decaying algorithm maintains a fixed exploration rate of $\epsilon = 0.1$ throughout all 94,942 impressions, meaning that it made random arm selections at the same rate, even after the algorithm had accumulated sufficient information to exploit reliably. As a result of this persistent exploration, the algorithm achieved a higher average regret (0.0536 vs 0.0480) and lower total reward (58178 vs 58570) as compared to the decaying algorithm.

The decaying algorithm begins with an exploration rate of $\epsilon = 0.3$, which gradually reduces to a floor of $\epsilon = 0.05$. As the algorithm shifts from exploration towards exploitation as the rounds accumulate, it was able to achieve a better CTR of 0.6169 (compared to 0.6128), and a reduced total regret of 10.5%, relative to the non-decaying algorithm (4553.9 vs 5088.9). From this, we are able to infer that while early exploration is valuable, sustained random exploration is wasteful once the algorithm has learned stable arm preferences.

This relationship is illustrated in Figure 5. Although both ϵ -greedy algorithms grow approximately linearly throughout the simulation, the non-decaying algorithm accumulates regret at a consistently steeper rate. As neither curve shows signs of flattening, we can infer that both ϵ -greedy algorithms continued to incur meaningful regret even in the later rounds.

5.2.3. LINEAR THOMPSON SAMPLING

Linear Thompson Sampling accumulated the highest cumulative regret of all four algorithms (6920.1 by the final round), underperforming even the non-decaying ϵ -greedy baseline. The average regret plot (Figure 6) shows that while Linear Thompson Sampling converges downwards over time, it does so at a considerably slower rate as compared to the other three algorithms.

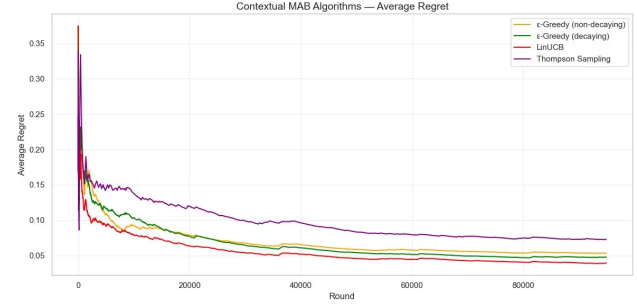


Figure 6. Average Regret of MAB Algorithms

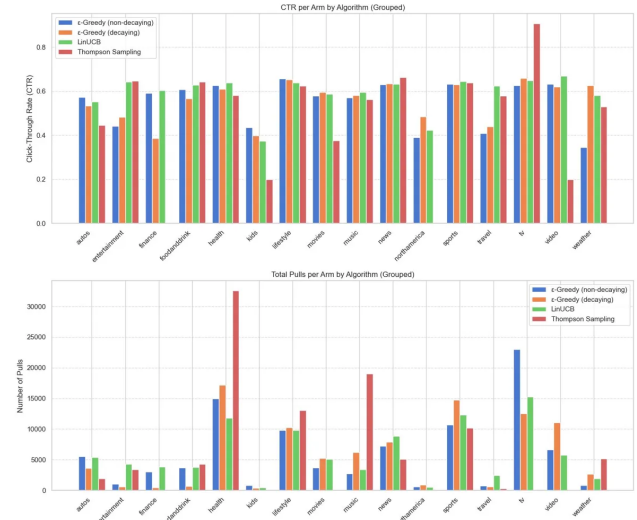


Figure 7. Arm Level Analyses (CTR and Total Pulls) of MAB Algorithms

Arm-level analyses (Figure 7) explain this underperformance. Unlike the other three algorithms, which distribute pulls relatively broadly, Linear Thompson Sampling concentrates arm pulls across selected categories. The algorithm pulled *Health* 32,546 times and *Music* 19,029 times, and allocated near-zero pulls to *Finance*, *Kids*, *Movies*, and *Northamerica*. This suggests that early stochastic samples from the posterior had produced high scores for *Health* and *Music*, even before their estimates were reliable. As a result, the algorithm over-committed to these arms. Despite pulling *health* the most, the Linear Thompson Sampling algorithm obtained the lowest CTR for that arm (0.5817) compared to the other three algorithms. This confirms that the algorithm's early commitments to the *Health* category were based on noise rather than a genuine signal.

Interestingly, despite only pulling the *TV* category only 85 times, Linear Thompson Sampling achieved the highest CTR of any algorithm (0.9059). This may suggest that

although it had identified a strong signal in that arm, it failed to exploit it. Instead, it continued to over-commit to *Health* and *Music*. Similarly, as a result of their near-zero pulls, *Finance* and *Northamerica* achieved CTRs of 0.0000. This suggests that the algorithm never gathered enough data to obtain reliable estimates for these arms. This demonstrates a key failure mode of posterior sampling with low variance, whereby early noisy estimates can result in irreversible over-exploitation, even before the algorithm has sufficient data to correct course.

5.2.4. COMPARISON ACROSS ALL ALGORITHMS

LinUCB substantially outperformed the three other algorithms across all metrics. It achieved the highest total reward of 59537 and the lowest cumulative regret of 3752.2. The key difference between these algorithms lies in how each algorithm handles exploration. The ε -greedy algorithms approach exploration in a random manner, selecting an arm uniformly at random from all available arms, with probability ε (Tanković & Šajina, 2025). Although Linear Thompson Sampling was more theoretically principled, the algorithm suffered from over-concentration due to noisy posterior estimates under low variance ($v = 0.1$) during the early rounds. On the other hand, the LinUCB algorithm uses its confidence bound ($\alpha\sqrt{x^T A^{-1}x}$) (Li et al., 2010) to direct exploration towards arms where uncertainty is high, and the predicted reward is promising. LinUCB’s principled uncertainty-driven exploration allows it to learn arm values more efficiently and hence shift to exploitation sooner.

This is observed in the cumulative regret plot (Figure 5), where LinUCB’s regret curve grows at a noticeably slower, and increasingly sub-linear rate compared to the other algorithms. This suggests that the LinUCB algorithm was converging toward better arm selections over time. This relationship is similarly observed in the average regret plot (Figure 6). Notably, LinUCB’s average regret stabilises at the lowest value amongst the four algorithms, at approximately 0.04, by the end of the simulation.

The declining and flattening of the average regret curves (Figure 6) over time indicate that all four algorithms learn from experience, hence progressively improving arm selection as more impressions are observed. However, the values at which each curve stabilises differ meaningfully. While LinUCB and the ε -greedy algorithms converge to a stable performance by approximately round 50,000, the Linear Thompson Sampling curve continues to decline more gradually. This suggests that the Linear Thompson Sampling algorithm required more rounds to reach a comparable steady state.

Arm-level analyses (Figure 7) further illustrate LinUCB’s efficiency. Compared to the ε -greedy algorithms, LinUCB allocates substantially fewer pulls to high-volume but low-

yield arms such as *Health*. Despite this, LinUCB was still able to achieve competitive or superior CTR for those arms. LinUCB also demonstrated notably stronger performance for the *Travel* category (0.6247 CTR with 2417 pulls) compared to Linear Thompson Sampling’s near-neglect of it (264 pulls, 0.5795 CTR). This further illustrates how LinUCB’s structured exploration can lead to more balanced and effective arm coverage. The relative arm rankings according to CTR for the three non-Thompson models remain broadly stable, with *Lifestyle*, *News*, *Sports*, and *TV* consistently attracting high CTRs, while *Kids* and *Northamerica* remain low. This suggests that the reward signal, which was captured by the logistic regression oracle, was reasonably stable across the different exploration strategies.

5.3. Key Insights

Exploration strategies which are uncertainty-aware outperform random exploration. All four MAB algorithms use per-arm context vectors, which incorporate user history features and semantic similarity. LinUCB’s consistent outperformance as compared to the ε -greedy algorithms confirms that directing exploration based on contextual uncertainty produces meaningfully better recommendations as compared to random exploration strategies. This is even so when the exploration rates are well-tuned.

Exploration scheduling affects the performance of ε -greedy algorithms. The 10.5% reduction in total regret by the decaying algorithm as compared to the non-decaying algorithm demonstrates that tapering exploration over time is beneficial, especially once stable arm preferences have been learned. However, both algorithms are still limited by their blind approach to exploration.

Posterior sampling is sensitive to hyperparameters at initialisation. The underperformance of Linear Thompson Sampling relative to the other algorithms highlights its sensitivity to the posterior variance parameter v . At $v = 0.1$, the samples were insufficiently diverse in early rounds. This led to the premature over-exploitation of arms whose estimates were still unreliable. In this aspect, an adaptive variance schedule which balances high exploration in the earlier rounds and high exploitation in later rounds may produce substantially different results (Daniel et al., 2018).

6. Limitations and Future Work

While the results show a clear performance hierarchy across the four algorithms, two aspects of the experimental setup limit how these findings should be interpreted.

First, the evaluation is conducted entirely in simulation rather than with real user interaction. Although the reward model is trained on an earlier time period and the bandits are evaluated only on subsequent, non-overlapping impres-

sions, the bandits still do not observe actual clicks. Rewards are generated by a logistic regression model with modest predictive power (AUC of 0.6598 and precision of 0.02 on clicked articles), so any systematic prediction errors in the model are carried through the simulation. The reported CTR and regret figures, therefore, reflect performance relative to the reward model rather than real users. Applying inverse propensity scoring or doubly-robust estimation could help separate true user preferences from artefacts of the simulation, and validating these findings in a live recommendation setting remains an important next step.

Second, defining arms at the category level is a coarse abstraction that limits personalisation, as users with different subcategory interests within the same category are treated identically. A hierarchical bandit that first selects a category and then a subcategory would allow finer-grained recommendations without an unmanageably large action space. The linear structure of LinUCB further constrains the model’s ability to capture nonlinear user-category relationships. Replacing it with a nonlinear alternative such as NeuralUCB could better capture complex interactions, particularly if richer features like article recency or text embeddings from article titles were introduced.

7. Conclusion

This study compared four contextual multi-armed bandit algorithms for category-level news recommendation on the MIND dataset, evaluating them on 94,942 held-out impressions that the reward model was never trained on. LinUCB achieved the highest CTR (0.6271) and lowest average regret (0.0395), accumulating 26% less total regret than the non-decaying baseline. The decaying ϵ -greedy variant reduced regret by 11% over its non-decaying counterpart, confirming that tapering exploration over time outperforms a fixed exploration rate. Linear Thompson Sampling underperformed all other algorithms, likely due to sensitivity to its variance hyperparameter rather than a fundamental shortcoming of posterior sampling. These results show that uncertainty-aware exploration, as implemented by LinUCB, produces meaningfully better recommendations than random exploration strategies, even when all algorithms share the same contextual features. However, the findings are bound by the fidelity of the offline simulation, and validating these conclusions in a live recommendation setting remains an important next step.

References

- Agrawal, S. and Goyal, N. Thompson sampling for contextual bandits with linear payoffs. In *Proceedings of the 30th International Conference on Machine Learning*, pp. 127–135, 2013. URL <https://proceedings.mlr.press/v28/agrawal13.pdf>.
- Daniel, J. R., Benjamin, V. R., Abbas, K., Ian, O., and Zheng, W. A tutorial on thompson sampling. *Foundations and Trends® in Machine Learning*, 11(1):1–99, 2018.
- Li, L., Chu, W., Langford, J., and Schapire, R. E. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th International Conference on World Wide Web*, pp. 661–670. Association for Computing Machinery, 2010. doi: 10.1145/1772690.1772758. URL <https://arxiv.org/abs/1003.0146>.
- Magesh, A. Thompson sampling for contextual bandits with linear payoffs. Course project report, University of Illinois Urbana-Champaign, May 2019. URL https://courses.grainger.illinois.edu/ece543/sp2019/projects/A_Magesh.pdf.
- Tanković, N. and Šajina, R. Scalable and interpretable contextual bandits: A literature review and retail offer prototype. In *International Conference on Data Analytics & Management*, pp. 527–538. Springer, 2025.
- Wu, F., Qiao, Y., Chen, J.-H., Wu, C., Qi, T., Lian, J., Liu, D., Xie, X., Gao, J., Wu, W., and Zhou, M. MIND: A large-scale dataset for news recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 3597–3606, 2020.

A. Appendix

Link to Github: <https://github.com/Hua-Swen/BT5153-Multi-Armed-Bandits>